

OPERATIONS GUIDE

ReVue-Remote Fresh Server Installation

A complete installation guide for recorded cloud video, Remote Live streaming, secure public access, persistent storage, commissioning, and verification.

AUDITED AGAINST

ReVue-Remote net10.0 application
MediaMTX RTMP/HLS integration
FFmpeg video-processing pipeline
Nginx reverse-proxy requirements

Prepared 5 October 2026 | Version 1.0

1. Installation at a glance

This guide describes a fresh production installation of the current ReVue-Remote application. Follow the sections in order to prepare the host, install the runtime and media services, configure persistent storage, secure public access, and commission the system.

Installation outcome

A completed installation provides the ReVue-Remote web application, recorded-video storage and processing, Remote Live ingest and playback, HTTPS access, automatic service startup, and a verified backup baseline.

Core production stack

Component	Needed for	Requirement
ASP.NET Core 10	All deployments	Install the ASP.NET Core 10 runtime for the current framework-dependent publish, or deploy a self-contained Linux build.
FFmpeg + ffprobe	All deployments	Required for upload inspection, MP4 remuxing, thumbnails, conversions, and Live event capture. FFmpeg must include libx264.
Nginx + TLS	Public deployment	Terminates HTTPS, provides HTTP/2, handles long-lived event streams, and proxies to Kestrel on loopback.
Persistent storage	All deployments	Stores users, Rink IDs, encryption keys, uploads, videos, playback state, and Live event files.
MediaMTX	Remote Live only	Accepts RTMP publishers and produces rolling fragmented-MP4 HLS on loopback.
systemd	Recommended	Starts ReVue-Remote and MediaMTX at boot and restarts them after failure.

Recommended host profile

Resource	Starting point	Planning note
Operating system	Ubuntu LTS	The application is cross-platform, but the documented reference deployment assumes Linux, Nginx, and systemd.
CPU	2-4 modern cores	Video conversions use two FFmpeg threads and run one job at a time. Live event remuxing is lighter than encoding.
Memory	4 GiB minimum; 8 GiB preferred	Increase for many simultaneous viewers, Live feeds, uploads, or management previews.
Storage	Local SSD	Size for the media library plus temporary processing space and Live recordings.
Network	Size by viewer count	Worst-case application video traffic is capped at 30 Mbps per viewer, plus about 6 Mbps per active recommended RTMP feed.

2. Host preparation and packages

Install only the runtime on a production host if the application is published elsewhere. The SDK is necessary only when building on the server.

Ubuntu package installation

```
sudo apt update
sudo apt install -y \
  aspnetcore-runtime-10.0 \
  ffmpeg nginx curl openssl ca-certificates
```

Verification

```
dotnet --list-runtimes
ffmpeg -version
ffprobe -version
ffmpeg -hide_banner -encoders | grep libx264
nginx -v
```

Runtime choice

The project targets net10.0. A framework-dependent deployment needs Microsoft.AspNetCore.App 10.x. Installing only dotnet-runtime-10.0 is insufficient because it omits ASP.NET Core.

Server preparation checklist

- [] Create a dedicated, non-login service account named revue.
- [] Create /opt/revue-remote for the published application.
- [] Create /var/lib/revue-remote on persistent storage.
- [] Create /etc/revue-remote for private environment and MediaMTX configuration.
- [] Grant the revue account read access to the application and configuration.
- [] Grant the revue account full read/write/create/rename/delete access to StorageRoot.
- [] Set the host clock and enable NTP; playback leases and security timing use UTC timestamps.
- [] Create the public DNS records for the production hostname.
- [] Obtain a valid TLS certificate for the public hostname.

Suggested account and directories

```
sudo useradd --system --home /var/lib/revue-remote \
  --shell /usr/sbin/nologin revue
sudo mkdir -p /opt/revue-remote /var/lib/revue-remote \
  /etc/revue-remote
sudo chown -R revue:revue /opt/revue-remote /var/lib/revue-remote
sudo chmod 750 /var/lib/revue-remote
sudo chmod 750 /etc/revue-remote
```

HTTPS is a functional requirement

The remote viewer uses a service worker and browser Cache Storage to retain video ranges for replay. Browsers provide those APIs only in a secure context, except for localhost. Public production use therefore requires HTTPS, not plain HTTP. HTTPS also protects administrator cookies, Rink IDs, and video traffic.

3. Network, firewall, and security

Expose only the ports that participate in the current production data path. Keep Kestrel, MediaMTX HLS, and the MediaMTX API on loopback.

Port	Bind/exposure	Purpose	Required
22/TCP	Restricted admin IPs	SSH administration and deployment.	Operational
80/TCP	Public	HTTP-to-HTTPS redirect and ACME certificate renewal.	Recommended
443/TCP	Public	Viewer, admin, VRO, uploads, HLS, SSE, and video playback.	Yes
1935/TCP	Encoder IPs or public	RTMP publishing for Remote Live.	Live only
5080/TCP	127.0.0.1	Internal ASP.NET Core/Kestrel listener.	Yes
8888/TCP	127.0.0.1	MediaMTX HLS consumed by ReVue-Remote.	Live only
9997/TCP	127.0.0.1	MediaMTX status and publisher-control API.	Live only

Public exposure Publish the HTTPS application endpoint on TCP 443 and the Remote Live RTMP ingest endpoint on TCP 1935. Bind the application, HLS service, and MediaMTX control API to loopback as shown in the table.

Runtime traffic

- Public clients and VRO connect to the HTTPS hostname on 443.
- An encoder publishes to `rtmp://HOST/RINKID` on TCP 1935.
- ReVue-Remote reads the MediaMTX rolling HLS feed through `127.0.0.1:8888`.
- ReVue-Remote reads MediaMTX status and can disconnect publishers through `127.0.0.1:9997`.
- MediaMTX authenticates publishers and readers through ReVue-Remote at `127.0.0.1:5080/api/internal/mediamtx-auth`.

Security requirements

- [] Restrict SSH to trusted administrator IP addresses and key authentication.
- [] Keep ports 5080, 8888, and 9997 inaccessible from the Internet.
- [] Block public access to `/api/internal/mediamtx-auth` at the reverse proxy.
- [] Store the shared MediaMTX authentication token only in root/service-readable files.
- [] Restrict RTMP 1935 to known encoder source IPs when practical.
- [] Use a VPN or separately configure encrypted publishing if RTMP crosses an untrusted network; the supplied RTMP configuration is plaintext.
- [] Route RTMP directly to the origin or through infrastructure that explicitly supports RTMP transport.
- [] Apply operating-system, .NET, Nginx, FFmpeg, and MediaMTX security updates under change control.

Reverse proxy placement

The application currently trusts X-Forwarded-For and X-Forwarded-Proto only when the proxy address is loopback. Run Nginx on the same host. A proxy in a container, on another host, or behind a load balancer requires an explicit trusted-proxy code/configuration change.

4. Persistent storage and backups

Configure an absolute storage path. The default relative path is suitable for development, but a production deployment should keep application binaries and persistent data separate.

```
ReVueRemote__StorageRoot=/var/lib/revue-remote
```

Directory contents

```
/var/lib/revue-remote/
|-- _system/
|   |-- users.json
|   |-- keys.json
|   |-- rink-id-access.json
|   |-- data-protection/
|-- RINKID/
|   |-- videos/
|   |-- uploads/
|   |-- live/
|   |-- videos.json
|   |-- playback.json
```

Critical backup rule

Back up the entire StorageRoot, including _system/data-protection. Those data-protection keys are needed to decrypt stored Live publish passwords. Treat the media folders, JSON state, and encryption keys as one backup set.

Capacity rules enforced by the application

Rule	Current behavior
Maximum input video	10 GiB per uploaded source file.
Processing workspace	Allow approximately twice the source size plus a 512 MiB safety margin.
Storage ceiling	ReVue uses at most approximately 90% of the filesystem space available to it.
Live storage	Live event segments and archive chunks count against the same storage ceiling.
Upload staging	Resumable uploads use 8 MiB chunks and temporary assembly/processing files.
Filesystem behavior	The service requires reliable file locking, capacity reporting, atomic rename, delete, and Range-friendly reads.

Sizing example

A maximum-size 10 GiB upload can require more than 20.5 GiB of temporary free space while its source and processed output coexist. This space is needed in addition to all existing recorded media and Live event data.

Backup policy

- Take filesystem-level backups of the complete StorageRoot and keep at least one off-host copy.
- Back up the private MediaMTX configuration and the ReVueRemote__MediaAuthToken environment file.
- Schedule filesystem snapshots during a quiet period with uploads and Live recordings paused.
- For the cleanest application-consistent backup, stop ReVue-Remote before copying JSON state and data-protection keys.
- Test restoration, not only backup creation.
- Keep each running ReVue-Remote instance on its own dedicated StorageRoot.

5. Application and MediaMTX configuration

Store production settings in a root-readable environment file rather than in the published application directory.

`/etc/revue-remote/revue-remote.env`

```
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_URLS=http://127.0.0.1:5080

ReVueRemote__StorageRoot=/var/lib/revue-remote
ReVueRemote__FFmpegPath=/usr/bin/ffmpeg
ReVueRemote__FFprobePath=/usr/bin/ffprobe

ReVueRemote__MediaMtxHlsBase=http://127.0.0.1:8888
ReVueRemote__MediaMtxApiBase=http://127.0.0.1:9997
ReVueRemote__MediaAuthToken=REPLACE_WITH_RANDOM_64_HEX_TOKEN

sudo mkdir -p /etc/revue-remote
sudo touch /etc/revue-remote/revue-remote.env
sudo chown root:revue /etc/revue-remote/revue-remote.env
sudo chmod 640 /etc/revue-remote/revue-remote.env
openssl rand -hex 32
```

MediaMTX

Use the repository's `mediamtx.live.example.yml` as the baseline, replace the token, and disable unused protocols. Select a tested MediaMTX 1.x release, record the installed version in the server build notes, and retain the installer with the deployment package.

```
logLevel: info
authMethod: http
authHTTPAddress: http://127.0.0.1:5080/api/internal/mediamtx-auth?token=TOKEN
authHTTPExclude:
  - action: api

api: yes
apiAddress: 127.0.0.1:9997

rtmp: yes
rtmpAddress: :1935
rtsp: no

hls: yes
hlsAddress: 127.0.0.1:8888
hlsVariant: fmp4
hlsSegmentDuration: 1s
hlsSegmentCount: 20
hlsAlwaysRemux: yes

webrtc: no
srt: no
paths:
  all_others:
    source: publisher
```

Shared secret

The token in MediaMTX `authHTTPAddress` must exactly match `ReVueRemote__MediaAuthToken`. Generate it with `openssl rand -hex 32`. Do not expose the callback URL or token through the public reverse proxy.

Live encoder profile

The expected publisher is 1280x720 progressive H.264/AVC yuv420p at constant 60000/1001 fps, 60-frame GOP, no B-frames, approximately 6000 kbps CBR, with AAC-LC stereo audio at 48 kHz and approximately 128 kbps. Publish to `rtmp://HOST/RINKID` and append URL-encoded publisher credentials as configured for the Live Rink ID.

6. Publish, install, and run the application

Upload the complete output of dotnet publish. Do not deploy only the DLL or copy the raw project directory.

Option A - framework-dependent deployment

```
dotnet publish ReVue-Remote/ReVue-Remote.csproj \
  -c Release \
  -o artifacts/ReVue-Remote
```

This is the repository's current publish model. The target server must have ASP.NET Core Runtime 10 installed. The output must include ReVue-Remote.dll, its .deps.json and .runtimeconfig.json files, appsettings.json, the full wwwroot directory, and all other publish output.

Option B - self-contained Linux deployment

```
dotnet publish ReVue-Remote/ReVue-Remote.csproj \
  -c Release -r linux-x64 --self-contained true \
  -o artifacts/ReVue-Remote-linux-x64
```

A self-contained build removes the server's .NET runtime dependency. FFmpeg, Nginx, MediaMTX, storage, TLS, and service configuration are still required.

Application systemd service

```
[Unit]
Description=ReVue-Remote cloud media service
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=revue
Group=revue
WorkingDirectory=/opt/revue-remote
EnvironmentFile=/etc/revue-remote/revue-remote.env
ExecStart=/usr/bin/dotnet /opt/revue-remote/ReVue-Remote.dll
Restart=always
RestartSec=5
KillSignal=SIGINT
TimeoutStopSec=30
LimitNOFILE=65536
NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

For a self-contained publish, replace ExecStart with the published executable. Ensure it is executable with chmod 750.

Enable and inspect

```
sudo systemctl daemon-reload
sudo systemctl enable --now revue-remote
sudo systemctl status revue-remote --no-pager
sudo journalctl -u revue-remote -n 100 --no-pager
curl -fsS http://127.0.0.1:5080/api/health
```

Single instance only

Run one ReVue-Remote application instance per deployment. Viewer notifications and subscription state are distributed in memory. Horizontal scaling would require shared pub/sub and additional coordination.

7. Nginx, HTTPS, and HTTP/2

The reverse proxy must preserve long-lived server-sent event connections, HTTP Range requests, streaming responses, large uploads, the original client address, and the original HTTPS scheme.

Required behavior

- Listen on 443 with a valid certificate and HTTP/2 enabled.
- Redirect public port 80 to HTTPS and allow certificate renewal.
- Proxy to `http://127.0.0.1:5080` using HTTP/1.1 upstream.
- Pass Host, X-Real-IP, X-Forwarded-For, and X-Forwarded-Proto.
- Disable response buffering for text/event-stream and media streaming.
- Disable request buffering for uploads.
- Allow request bodies up to 11 GiB for full backward compatibility with the legacy multipart endpoint.
- Permit long-lived reads and writes.
- Block the private MediaMTX authentication endpoint from public clients.

Reference Nginx server block

```
server {
    listen 80;
    server_name remote.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name remote.example.com;

    ssl_certificate     /etc/letsencrypt/live/remote.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/remote.example.com/privkey.pem;

    client_max_body_size 11g;

    location = /api/internal/mediamtx-auth { deny all; }

    location / {
        proxy_pass http://127.0.0.1:5080;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_request_buffering off;
        proxy_buffering off;
        proxy_read_timeout 24h;
        proxy_send_timeout 24h;
    }
}
```

Use the HTTP/2 syntax supported by the installed Nginx version. Newer versions may use a separate `http2 on` directive.

Validate the proxy

```
sudo nginx -t
sudo systemctl reload nginx
curl -fsS https://remote.example.com/api/health
curl -sS -o /dev/null -w '%{http_version}' \
--http2 https://remote.example.com/api/health
```

The final command should print 2. HTTP/2 matters because several viewer tabs can each maintain a long-lived EventSource connection to the same host.

8. Fresh installation runbook

Objective

Commission a new ReVue-Remote production host with persistent storage, secure HTTPS access, recorded-video processing, Remote Live ingest, automatic startup, monitoring, and an initial verified backup.

- 1** Provision the host
Create an Ubuntu LTS server with the recommended CPU, memory, SSD capacity, static public address, correct time zone, and NTP synchronization.
- 2** Configure DNS and TLS
Point the production hostname to the server, allow time for DNS propagation, and obtain a trusted certificate for that hostname.
- 3** Install production packages
Install ASP.NET Core Runtime 10, FFmpeg, ffprobe, Nginx, curl, OpenSSL, CA certificates, and the selected MediaMTX release. Record all installed versions.
- 4** Create the service identity
Create the revue system account plus `/opt/revue-remote`, `/var/lib/revue-remote`, and `/etc/revue-remote` with the ownership and permissions specified in this guide.
- 5** Publish and install ReVue-Remote
Build the Release publish output, upload the complete output to `/opt/revue-remote`, and confirm that `wwwroot`, `settings`, `runtime` metadata, and application binaries are present.
- 6** Configure application storage
Create the environment file with the loopback Kestrel address, absolute StorageRoot, FFmpeg paths, MediaMTX addresses, and a newly generated 64-hex-character shared token.
- 7** Configure MediaMTX
Install the reviewed `mediamtx.live.example.yml` baseline, insert the matching shared token, bind HLS and the API to loopback, and enable RTMP publishing on TCP 1935.
- 8** Install systemd services
Install service units for ReVue-Remote and MediaMTX, enable them at boot, configure restart behavior, and verify that the private files are readable by their service identities.
- 9** Configure Nginx and the firewall
Install the HTTPS server block, HTTP redirect, proxy headers, streaming timeouts, upload limits, private-path block, and HTTP/2 support. Apply the documented inbound firewall rules.
- 10** Start and verify dependencies
Start ReVue-Remote, start MediaMTX, and reload Nginx. Verify loopback listeners and health endpoints before testing the public hostname.
- 11** Complete first-run setup
Sign in with the temporary administrator credentials, change the password immediately, create the required Regular accounts and Rink IDs, and configure a Live Rink ID where applicable.
- 12** Commission and protect the service
Run the complete acceptance checklist, confirm monitoring and log retention, create the first full StorageRoot and configuration backup, and test that the backup can be read.

Installation record

Save the package versions, publish build identifier, configuration checksums, public hostname, certificate renewal method, firewall rules, service-unit files, and backup location as the server's commissioning record.

9. Acceptance and verification checklist

Infrastructure

- [] ReVue-Remote, MediaMTX, and Nginx are enabled and active in systemd.
- [] `dotnet --list-runtimes` shows Microsoft.AspNetCore.App 10.x, unless self-contained.
- [] FFmpeg, fprobe, and libx264 checks pass.
- [] Kestrel listens only on 127.0.0.1:5080.
- [] MediaMTX HLS listens only on 127.0.0.1:8888.
- [] MediaMTX API listens only on 127.0.0.1:9997.
- [] Only 80, 443, restricted 22, and required 1935 are externally reachable.
- [] The public certificate is trusted and the hostname is correct.
- [] Public `/api/health` returns status ok and negotiates HTTP/2.
- [] Public `/api/internal/mediamtx-auth` is blocked.

Recorded-video workflow

- [] The temporary administrator account can sign in and its password has been changed.
- [] Required Regular accounts and Rink IDs have been created.
- [] A small supported upload completes and survives application restart.
- [] A thumbnail is generated.
- [] A video can play, seek, and return correct HTTP Range responses.
- [] A 1080-to-720 or high-frame-rate conversion completes successfully.
- [] VRO can select and use a RemoteRecorded video.

Remote Live workflow

- [] A publisher can connect to TCP 1935 with the configured Rink ID and password.
- [] The Rink ID manager reports the feed as active.
- [] The browser and VRO show the rolling Live preview.
- [] Record starts a new event and produces event HLS segments.
- [] Stop finalizes replay and limits it to the recorded duration.
- [] Next Competitor removes the previous event and returns to the rolling feed.
- [] A refreshed viewer joins the correct current event without old footage.
- [] Multiple roles receive communication and playback updates over SSE.

Operations

- [] The storage screen reports the correct filesystem and capacity.
- [] `journalctl` contains no permission, data-protection, FFmpeg, HLS, or MediaMTX API errors.
- [] The complete StorageRoot and private configuration are included in a new backup.
- [] A restore test or checksum comparison has been completed.
- [] Monitoring alerts on service failure, low disk space, certificate expiry, and port availability.

10. Operational notes and quick diagnostics

Symptom	Likely cause	First checks
App will not start	Missing ASP.NET runtime, unreadable files, or invalid environment file	dotnet --list-runtimes; systemctl status; journalctl -u revue-remote
Uploads fail	Nginx body limit/buffering, permissions, or insufficient temporary disk	Nginx error log; StorageRoot ownership; df -h; storage screen
No thumbnails/conversion	FFmpeg or ffprobe missing, or FFmpeg lacks libx264	Absolute binary paths; ffmpeg -encoders; service PATH
Login loops after HTTPS	Forwarded protocol not passed/trusted or old data-protection keys missing	X-Forwarded-Proto; loopback Nginx; restored data-protection directory
SSE disconnects	Proxy buffering, short read timeout, or HTTP/1.1 connection limits	proxy_buffering off; 24h timeout; public HTTP/2 test
RTMP rejected	Wrong Rink type/password or mismatched MediaAuthToken	MediaMTX logs; Live Rink ID; callback token equality
Live preview absent	MediaMTX HLS unavailable or wrong loopback base	curl 127.0.0.1:8888; MediaMtxHlsBase; publisher status
Live status unavailable	MediaMTX API unavailable or wrong base	curl 127.0.0.1:9997/v3/paths/list; MediaMtxApiBase
Storage shown unavailable	Filesystem capacity cannot be inspected or permissions deny traversal	Mount visibility; DriveInfo-compatible mount; execute/read permission

Fresh-install administrator

Change immediately

A brand-new StorageRoot creates admin@admin.com with temporary password admin. Use it for the first sign-in, set a unique strong password immediately, and store the administrator credential in the approved password manager.

Source audit

- ReVue-Remote/ReVue-Remote.csproj - target framework and publish model.
- ReVue-Remote/Program.cs - HTTP pipeline, forwarded headers, upload limits, API endpoints, authentication, HLS proxy, SSE, and health check.
- ReVue-Remote/RemoteSessionStore.cs - storage tree, upload limits, capacity calculations, and state files.
- ReVue-Remote/VideoCanonicalizer.cs - FFmpeg/ffprobe dependencies, accepted codecs, thumbnails, and transcodes.
- ReVue-Remote/LiveStreamService.cs - Live HLS input, event remuxing, and Live storage.
- ReVue-Remote/LiveStreamStatusService.cs - MediaMTX API integration.
- ReVue-Remote/CloudAccountStore.cs - users, Rink IDs, passwords, and data-protection usage.
- ReVue-Remote/mediamtx.live.example.yml - RTMP/HLS/API listener and authentication settings.

External references

Microsoft - Install .NET on Ubuntu:

<https://learn.microsoft.com/en-us/dotnet/core/install/linux-ubuntu-install>

Microsoft - Host ASP.NET Core on Linux with Nginx:

<https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/linux-nginx>

MediaMTX - Install and start on boot:

<https://mediamtx.org/docs/kickoff/install>

<https://mediamtx.org/docs/features/start-on-boot>

End of guide